

C++ Codebook

This codebook contains commonly used algorithms and utilities optimized for competitive programming.

1. Basic Utilities

IO Speedup (cin, cout)

```
1 cin.sync_with_stdio(0);
2 cin.tie(0);
```

Buffered Output (Defer Flushing Until Program End)

```
1 ios::sync_with_stdio(false);
2 cin.tie(NULL);
```

Test With File

```
1 g++ main.cpp -o main
2 main < input.in > output.out
```

Write File

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 int main() {
6     std::ofstream outFile("example.txt");
7     if (!outFile.is_open()) {
8         std::cerr << "Error opening file for
9         ↳ writing!" << std::endl;
10        return 1;
11    }
12    outFile << "Hello, World!\n";
13    outFile << "This is a line in a text file.\n";
14    outFile << 123 << " " << 45.67 << "\n";
15    outFile.close();
16    return 0;
17 }
```

Read File

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 int main() {
6     std::ifstream inFile("example.txt");
7     if (!inFile.is_open()) {
8         std::cerr << "Error opening file for
9         ↳ reading!" << std::endl;
10        return 1;
11    }
12    std::string line;
13    while (std::getline(inFile, line)) {
14        std::cout << line << std::endl;
15    }
16    inFile.close();
17    return 0;
18 }
```

Greatest Common Divisor (GCD)

```
1 int GCD(int x, int y) {
2     if (!x) return y;
3     if (!y) return x;
4     while ((x %= y) && (y %= x));
5     return x + y;
6 }
```

Least Common Multiple (LCM)

```
1 int LCM(int num1, int num2) {
2     return ((num1 * num2) / GCD(num1, num2));
3 }
```

GCD and LCM in C++17

```
1 #include <numeric>
2 int main () {
3     int a = 48, b = 18;
4     int gcd = std::gcd(a, b);
5     int lcm = std::lcm(a, b);
6     return 0;
7 }
```

Priority Queue (std::priority_queue)

Description: Demonstrates the use of C++ built-in `std::priority_queue` (from `<queue>`), including default behavior (Max-Heap), built-in Min-Heap, and Custom Struct Sorting. - **Default:** Max-Heap (largest element at the top). - **Built-in Min-Heap:** Uses `std::greater` (smallest element at the top). Very common with `std::pair` in graph algorithms. - **Custom Struct Sorting:** Overload the `<` operator inside a custom struct.

```
1 #include <queue>
2 #include <vector>
3 #include <iostream>
4 #include <utility> // for pair
5
6 using namespace std;
7
8 // Struct with custom sorting rules
9 struct Item {
10     int id;
11     int priority;
12
13     // Custom sorting rule: sort by priority
14     ↳ ascending (min-heap)
15     // If priority is equal, sort by id ascending
16     ↳ (smaller id at the top)
17     // Note: Since priority_queue is a Max-Heap by
18     ↳ default, we invert the comparison logic.
19     bool operator<(const Item& other) const {
20         if (priority != other.priority) {
21             return priority > other.priority; //
22             ↳ Larger priority means "less" priority
23             ↳ in max-heap (so smaller comes to top)
24         }
25         return id > other.id; // Larger ID means
26         ↳ "less" priority (so smaller ID comes to
27         ↳ top)
28     }
29 };
30
31 void priorityQueueUsage() {
32     priority_queue<int> max_pq;
```

```

26 max_pq.push(10);
27 max_pq.push(30);
28 max_pq.push(20);
29 // max_pq.top() is 30 (largest element first)
30
31 priority_queue<int, vector<int>, greater<int>>
↪ min_pq;
32 min_pq.push(10);
33 min_pq.push(30);
34 min_pq.push(20);
35 // min_pq.top() is 10 (smallest element first)
36
37 priority_queue<pair<int, int>, vector<pair<int,
↪ int>>, greater<pair<int, int>>> pair_pq;
38 pair_pq.push({5, 101}); // {distance, node}
39 pair_pq.push({2, 102});
40 pair_pq.push({5, 100});
41 // pair_pq.top() is {2, 102} (smallest distance
↪ first, then smallest node index)
42
43 priority_queue<Item> custom_pq;
44 custom_pq.push({101, 5});
45 custom_pq.push({102, 5});
46 custom_pq.push({103, 2});
47 // custom_pq.top() is {103, 2} (priority 2), then
↪ {101, 5} (priority 5, smaller id first)
48 }

```

2. Prime Numbers

Count Primes (Sieve of Eratosthenes)

```

1 #include <vector>
2 #include <algorithm>
3
4 using namespace std;
5
6 int countPrimes(int N) {
7     if (N < 2) return 0;
8     vector<bool> isPrime(N + 1, true);
9     isPrime[0] = isPrime[1] = false;
10    for (int i = 2; i * i <= N; ++i) {
11        if (isPrime[i]) {
12            for (int j = i * i; j <= N; j += i) {
13                isPrime[j] = false;
14            }
15        }
16    }
17    return count(isPrime.begin(), isPrime.end(),
↪ true);
18 }

```

Prime Number Test

```

1 bool isPrime(long long n) {
2     if (n <= 1) return false;
3     if (n <= 3) return true;
4     if (n % 2 == 0 || n % 3 == 0) return false;
5     for (long long i = 5; i * i <= n; i += 6) {
6         if (n % i == 0 || n % (i + 2) == 0) {
7             return false;
8         }
9     }
10    return true;
11 }

```

List Primes (Sieve of Eratosthenes)

```

1 #include <vector>
2 using namespace std;
3 vector<int> sieve(int N) {
4     vector<bool> isPrime(N + 1, true);
5     isPrime[0] = isPrime[1] = false;
6     for (int i = 2; i * i <= N; ++i) {
7         if (isPrime[i]) {
8             for (int j = i * i; j <= N; j += i) {
9                 isPrime[j] = false;
10            }
11        }
12    }
13    vector<int> primes;
14    for (int i = 2; i <= N; ++i) {
15        if (isPrime[i]) {
16            primes.push_back(i);
17        }
18    }
19    return primes;
20 }

```

Prime Factorization

```

1 #include <vector>
2 #include <utility>
3 using namespace std;
4 vector<pair<long long, int>> primeFactorization(long
↪ long n) {
5     vector<pair<long long, int>> factors;
6     for (long long i = 2; i * i <= n; ++i) {
7         if (n % i == 0) {
8             int count = 0;
9             while (n % i == 0) {
10                count++;
11                n /= i; // Eliminate the factor
12            }
13            factors.push_back({i, count});
14        }
15    }
16    if (n > 1) {
17        factors.push_back({n, 1}); // Remaining prime
↪ factor
18    }
19    return factors;
20 }

```

3. String Processing

String Split (Regex)

Description: Splits a string into substrings based on a regular expression pattern delimiter. **Parameters:** - **str:** The target string to split. - **keys:** Regex pattern specifying the delimiters (default matches dashes - or commas ,). - **keep:** If true, retains the matched delimiters as separate tokens in the output. **Usage:** Ideal for parsing complex string inputs. Requires <regex>.

```

1 #include <vector>
2 #include <string>
3 #include <regex>
4
5 using namespace std;
6
7 vector<string> split(const string& str, string keys =
↪ "{-[,3]|,}", bool keep = false) {
8     vector<string> tokens;

```

```

9     regex re(keys);
10    sregex_token_iterator it;
11    if (keep) {
12        it = sregex_token_iterator(str.begin(),
↪ str.end(), re, {-1, 0});
13    } else {
14        it = sregex_token_iterator(str.begin(),
↪ str.end(), re, -1);
15    }
16    sregex_token_iterator end;
17    while (it != end) {
18        tokens.push_back(*it);
19        ++it;
20    }
21    return tokens;
22 }

```

```

37         if (j == m) {
38             result.push_back(i - j); // Found
↪ match at index (i - j)
39             j = lps[j - 1];
40         }
41     } else {
42         if (j != 0) {
43             j = lps[j - 1];
44         } else {
45             i++;
46         }
47     }
48 }
49 return result;
50 }

```

Knuth-Morris-Pratt (KMP) Substring Search

Description: High-performance pattern searching algorithm. Precomputes the Longest Prefix Suffix (LPS) table to skip redundant character comparisons, eliminating the need to backtrack the text index. **Time Complexity:** $O(N + M)$ where N is text length, M is pattern length. **Space Complexity:** $O(M)$ for the LPS table. **Usage:** Fast substring matching when brute-force $O(N \cdot M)$ is too slow.

```

1  #include <vector>
2  #include <string>
3
4  using namespace std;
5
6  // Build the Longest Prefix which is also a Suffix
↪ (LPS) table
7  vector<int> buildLPS(const string &pattern) {
8      int m = pattern.size();
9      vector<int> lps(m, 0);
10     int len = 0; // Length of the previous longest
↪ prefix suffix
11     for (int i = 1; i < m; ) {
12         if (pattern[i] == pattern[len]) {
13             len++;
14             lps[i] = len;
15             i++;
16         } else {
17             if (len != 0) {
18                 len = lps[len - 1]; // Backtrack in
↪ pattern
19             } else {
20                 lps[i++] = 0;
21             }
22         }
23     }
24     return lps;
25 }
26
27 // KMP search main function: Returns starting indices
↪ of matches
28 vector<int> KMPsearch(const string &text, const
↪ string &pattern) {
29     vector<int> result;
30     int n = text.size(), m = pattern.size();
31     if (m == 0) return result;
32     vector<int> lps = buildLPS(pattern);
33     int i = 0, j = 0; // i -> text index, j ->
↪ pattern index
34     while (i < n) {
35         if (text[i] == pattern[j]) {
36             i++; j++;

```

4. Computational Geometry

Closest Pair of Points (Divide & Conquer)

Description: Finds the closest pair of points in a 2D Euclidean plane using a divide-and-conquer strategy. 1. **Sort:** Sort all points by X-coordinates. 2. **Divide & Solve:** Recursively find the minimum distance in left and right halves. 3. **Combine:** Check points within a vertical strip of width $2d$ around the midline. Sort them by Y-coordinates and compare each point with at most 6 subsequent points. **Time Complexity:** $O(N \log N)$ **Space Complexity:** $O(N)$ **Usage:** Standard high-performance solution for closest-pair problems.

```

1  #include <vector>
2  #include <cmath>
3  #include <algorithm>
4  #include <limits>
5  #include <utility>
6
7  using namespace std;
8
9  struct Point {
10     double x, y;
11 };
12
13 // Calculate Euclidean distance between two points
14 double dist(const Point& a, const Point& b) {
15     return hypot(a.x - b.x, a.y - b.y);
16 }
17
18 // Brute force method for 3 or fewer points
19 double bruteForce(vector<Point>& points, int left,
↪ int right, pair<Point, Point>& bestPair) {
20     double minDist = numeric_limits<double>::max();
21     for (int i = left; i <= right; ++i) {
22         for (int j = i + 1; j <= right; ++j) {
23             double d = dist(points[i], points[j]);
24             if (d < minDist) {
25                 minDist = d;
26                 bestPair = {points[i], points[j]};
27             }
28         }
29     }
30     return minDist;
31 }
32
33 // Check the strip area for closer points crossing
↪ the divide boundary
34 double stripClosest(vector<Point>& strip, double d,
↪ pair<Point, Point>& bestPair) {
35     double minDist = d;

```

```

36     sort(strip.begin(), strip.end(), [](const Point&
↪ a, const Point& b) {
37         return a.y < b.y;
38     });
39     for (size_t i = 0; i < strip.size(); ++i) {
40         // Compare each point with next points inside
↪ the strip
41         for (size_t j = i + 1; j < strip.size() &&
↪ (strip[j].y - strip[i].y) < minDist; ++j)
↪ {
42             double newDist = dist(strip[i],
↪ strip[j]);
43             if (newDist < minDist) {
44                 minDist = newDist;
45                 bestPair = {strip[i], strip[j]};
46             }
47         }
48     }
49     return minDist;
50 }

51 // Recursive Divide & Conquer helper utility
52 double closestUtil(vector<Point>& pointsSortedByX,
53 ↪ int left, int right, pair<Point, Point>&
↪ bestPair) {
54     // Use brute force if points count is 3 or less
55     if (right - left <= 3) {
56         return bruteForce(pointsSortedByX, left,
↪ right, bestPair);
57     }
58     int mid = (left + right) / 2;
59     Point midPoint = pointsSortedByX[mid];
60     pair<Point, Point> leftPair, rightPair;
61     double dl = closestUtil(pointsSortedByX, left,
↪ mid, leftPair);
62     double dr = closestUtil(pointsSortedByX, mid +
↪ 1, right, rightPair);
63     double d = dl;
64     bestPair = leftPair;
65     if (dr < dl) {
66         d = dr;
67         bestPair = rightPair;
68     }
69     // Gather points inside vertical strip of width
↪ 2d
70     vector<Point> strip;
71     for (int i = left; i <= right; ++i) {
72         if (abs(pointsSortedByX[i].x - midPoint.x) <
↪ d) {
73             strip.push_back(pointsSortedByX[i]);
74         }
75     }
76     pair<Point, Point> stripPair;
77     double dStrip = stripClosest(strip, d,
↪ stripPair);
78     if (dStrip < d) {
79         d = dStrip;
80         bestPair = stripPair;
81     }
82     return d;
83 }

84 // Main entry function to find the closest pair of
↪ points
85 pair<Point, Point> findClosestPair(vector<Point>&
↪ points) {
86     vector<Point> pointsSortedByX = points;
87     sort(pointsSortedByX.begin(),
88 ↪ pointsSortedByX.end(), [](const Point& a, const
↪ Point& b) {
89         return a.x < b.x;
90 });

```

```

91     pair<Point, Point> bestPair;
92     closestUtil(pointsSortedByX, 0,
↪ pointsSortedByX.size() - 1, bestPair);
93     return bestPair;
94 }

```

5. Permutation Operations

Next Permutation (Array-based)

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Returns the lexicographically next permutation of
↪ the array.
7  vector<int> nextPermutation(vector<int> nums) {
8      int n = nums.size();
9      int i = n - 2;
10     // Find the first decreasing element from the
↪ right
11     while (i >= 0 && nums[i] >= nums[i + 1]) {
12         i--;
13     }
14     if (i >= 0) {
15         // Find the successor to nums[i] from the
↪ right
16         int j = n - 1;
17         while (nums[j] <= nums[i]) {
18             j--;
19         }
20         swap(nums[i], nums[j]);
21     }
22     // Reverse the remaining ascending suffix
23     reverse(nums.begin() + i + 1, nums.end());
24     return nums;
25 }

```

Permutation Rank (Lexicographical Rank)

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Binary Indexed Tree (Fenwick Tree) helper
7  struct FenwickTree {
8      int n;
9      vector<int> tree;
10     FenwickTree(int n) : n(n), tree(n + 1, 0) {}
11     void update(int i, int delta) {
12         for (; i <= n; i += i & -i) tree[i] += delta;
13     }
14     int query(int i) {
15         int sum = 0;
16         for (; i > 0; i -= i & -i) sum += tree[i];
17         return sum;
18     }
19 };
20
21 // Calculates the 1-based lexicographical rank of a
↪ permutation.
22 // Assumes unique elements. Returns rank modulo MOD
↪ (default is 1e9 + 7).
23 long long getPermutationRank(vector<int> nums, long
↪ long MOD = 1000000007) {
24     int n = nums.size();
25     if (n == 0) return 1;
26 }

```

```

27 // Precompute factorials modulo MOD
28 vector<long long> fact(n, 1);
29 for (int i = 1; i < n; ++i) {
30     fact[i] = (fact[i - 1] * i) % MOD;
31 }
32
33 // Coordinate compression to map values to [1, n]
34 vector<int> temp = nums;
35 sort(temp.begin(), temp.end());
36 vector<int> compressed(n);
37 for (int i = 0; i < n; ++i) {
38     compressed[i] = lower_bound(temp.begin(),
↪ temp.end(), nums[i]) - temp.begin() + 1;
39 }
40
41 FenwickTree bit(n);
42 for (int i = 1; i <= n; ++i) {
43     bit.update(i, 1);
44 }
45
46 long long rank = 1;
47 for (int i = 0; i < n; ++i) {
48     // Count how many unused elements to the
49     ↪ right are smaller than compressed[i]
50     int smallerCount = bit.query(compressed[i] -
51     ↪ 1);
52     rank = (rank + smallerCount * fact[n - 1 -
53     ↪ i]) % MOD;
54     // Mark current element as used
55     bit.update(compressed[i], -1);
56 }
57 return rank;
58 }

```

Get Permutation by Rank

Description: Takes an integer rank (and the size N) and returns the corresponding permutation array using the factorial number system (Lehmer code).

Time Complexity: $O(N^2)$ with linear scan, or $O(N \log N)$ with Fenwick Tree. **Space Complexity:** $O(N)$ **Usage:** Decodes a 0-based rank into the k -th lexicographic permutation of $\{0, 1, \dots, N-1\}$.

```

1 #include <vector>
2 #include <algorithm>
3
4 using namespace std;
5
6 // Returns the permutation corresponding to the given
7 ↪ 0-based rank.
8 // Elements are drawn from {0, 1, ..., n-1}.
9 vector<int> getPermutationByRank(int n, long long
10 ↪ rank) {
11     // Precompute factorials
12     vector<long long> fact(n, 1);
13     for (int i = 1; i < n; ++i) {
14         fact[i] = fact[i - 1] * i;
15     }
16
17     // Available elements (sorted)
18     vector<int> available;
19     for (int i = 0; i < n; ++i)
20     ↪ available.push_back(i);
21
22     vector<int> perm;
23     for (int i = n - 1; i >= 0; --i) {
24         // Determine which available element goes to
25         ↪ this position
26         int idx = rank / fact[i];
27         rank %= fact[i];

```

```

24 perm.push_back(available[idx]);
25 available.erase(available.begin() + idx);
26 }
27 return perm;
28 }

```

6. Math

Euler Totient Function

Description: Calculates Euler's totient function $\phi(x)$, which counts the number of positive integers strictly less than x that are coprime to x . **Time Complexity:** $O(\sqrt{x})$

Space Complexity: $O(1)$ **Usage:** Commonly used in number theory, particularly for finding modular multiplicative inverses and applying Euler's theorem.

```

1 #include <cmath>
2
3 int Phi(int x) {
4     if (x < 2) return 0;
5     int ret = x;
6     int sq = sqrt(x);
7     for (int p = 2; p <= sq; p++) {
8         if (x % p == 0) {
9             while (x % p == 0) x /= p;
10            ret -= ret / p;
11        }
12        if (x == 1) break;
13    }
14    if (x > 1) ret -= ret / x;
15    return ret;
16 }

```

Modular Power (Fast Exponentiation)

Description: Computes $a^b \pmod{p}$ efficiently using binary exponentiation (repeated squaring).

Time Complexity: $O(\log b)$ **Space Complexity:** $O(1)$ **Usage:** Core building block for number theory problems involving large exponents under a modulus.

```

1 // Computes a^b mod p
2 long long modpow(long long a, long long b, long long
3 ↪ p) {
4     long long res = 1;
5     a %= p;
6     while (b > 0) {
7         if (b & 1) res = res * a % p;
8         a = a * a % p;
9         b >>= 1;
10    }
11    return res;
12 }

```

Modular Inverse

Description: In the world of remainders modulo p (where p is a prime number), we cannot use division or fractions directly. By using Fermat's Little Theorem, the calculated "Output (a^{p-2})" acts as a perfect substitute for a . It allows us to turn a difficult division problem into a simple multiplication problem:

$$x \div a \equiv x \times a^{p-2} \pmod{p}$$

Time Complexity: $O(\log p)$ **Space Complexity:** $O(1)$ **Requires:** modpow function above.

```
1 // Computes a^{-1} mod p (p must be prime)
2 long long modInverse(long long a, long long p) {
3     return modpow(a, p - 2, p);
4 }
```

Combinations Modulo p ($nCr \pmod{p}$)

Description: Precomputes factorials and inverse factorials to answer $\binom{n}{r} \pmod{p}$ queries in $O(1)$ each after $O(N)$ preprocessing. **Time Complexity:** $O(N)$ precomputation, $O(1)$ per query. **Space Complexity:** $O(N)$ **Requires:** modpow and modInverse functions above.

```
1 #include <vector>
2 using namespace std;
3
4 const int MAXN = 1e6 + 5;
5 const long long MOD = 1e9 + 7;
6
7 vector<long long> fact(MAXN), inv_fact(MAXN);
8
9 void precompute(int n) {
10     fact[0] = 1;
11     for (int i = 1; i <= n; ++i) {
12         fact[i] = fact[i - 1] * i % MOD;
13     }
14     inv_fact[n] = modInverse(fact[n], MOD);
15     for (int i = n - 1; i >= 0; --i) {
16         inv_fact[i] = inv_fact[i + 1] * (i + 1) %
17         MOD;
18     }
19 }
20
21 // Returns C(n, r) mod MOD
22 long long C(int n, int r) {
23     if (r < 0 || r > n) return 0;
24     return fact[n] % MOD * inv_fact[r] % MOD *
25     inv_fact[n - r] % MOD;
26 }
```

7. Range Queries

Difference Array

Description: Efficiently applies multiple range addition updates $[L, R]$ by modifying only the boundaries of a difference array. A prefix sum is then used to reconstruct the final array values. **Returns:** The minimum coverage value across all positions after applying all updates. **Time Complexity:** $O(M + N)$ where M is the number of updates and N is the array size. **Space Complexity:** $O(N)$ for the difference array. **Usage:** Ideal for scenarios requiring many range updates followed by a single full-array query or state evaluation.

```
1 #include <iostream>
2 #include <vector>
3 #include <climits>
4
5 using namespace std;
6
7 long long differenceArray() {
```

```
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long N, M;
12     if (!(cin >> N >> M)) return -1;
13
14     // Size N+2 to prevent R+1 from exceeding
15     // boundary
16     vector<long long> diff(N + 2, 0LL);
17
18     // Read interval [L, R] for each turret
19     for (int i = 0; i < M; i++) {
20         long long L, R;
21         cin >> L >> R;
22         // Difference array update: +1 at start L, -1
23         // at position after end R+1
24         diff[L] += 1;
25         if (R + 1 <= N) {
26             diff[R + 1] -= 1;
27         }
28     }
29
30     // Compute prefix-sum directly on diff array and
31     // track the minimum value
32     long long cover = 0; // Current
33     // cumulative coverage at the current position
34     long long answer = LLONG_MAX; // Stores the
35     // minimum coverage value
36
37     for (int pos = 1; pos <= N; pos++) {
38         cover += diff[pos]; // diff[pos] indicates
39         // the change in coverage at pos
40         // cover represents the actual coverage count
41         // at pos
42         if (cover < answer) {
43             answer = cover;
44         }
45     }
46
47     // Under normal circumstances, answer is at least
48     // 0.
49     // Return the answer.
50     return answer;
51 }
```

Fenwick Tree (Binary Indexed Tree)

Description: Support Point Update, Prefix Sum Query, Range Sum Query **Complexity:** Update : $O(\log N)$, Query : $O(\log N)$, Memory : $O(N)$

```
1 struct BIT {
2     int n;
3     vector<long long> bit;
4
5     BIT(int _n) {
6         n = _n;
7         bit.assign(n + 1, 0);
8     }
9
10    void add(int idx, long long val) {
11        while (idx <= n) {
12            bit[idx] += val;
13            idx += idx & -idx;
14        }
15    }
16
17    long long sum(int idx) {
18        long long res = 0;
19
20        while (idx > 0) {
```

```

21         res += bit[idx];
22         idx -= idx & -idx;
23     }
24
25     return res;
26 }
27
28 long long query(int l, int r) {
29     return sum(r) - sum(l - 1);
30 }
31 };

```

Note: - lowbit[i] returns the size of the range managed by index i:

```
1 lowbit(x) = x & -x
```

- tree[i] stores the sum for the range managed by index

8. Graph Algorithms

Dijkstra's Algorithm (Single-Source Shortest Path)

```

1 #include <vector>
2 #include <queue>
3 #include <climits>
4
5 using namespace std;
6
7 const long long INF = LLONG_MAX;
8
9 // adj[u] = list of {v, weight} representing edge u
10 // -> v with given weight
11 vector<long long> dijkstra(int n,
12 // vector<vector<pair<int, long long>>> adj, int
13 // src) {
14 //     vector<long long> dist(n + 1, INF);
15 //     dist[src] = 0;
16
17 // {distance, node}, min-heap by distance
18 // priority_queue<pair<long long, int>,
19 // vector<pair<long long, int>>, greater<>> pq;
20 // pq.push({0, src});
21
22 // while (!pq.empty()) {
23 //     auto [d, u] = pq.top();
24 //     pq.pop();
25
26 //     if (d > dist[u]) continue; // Outdated entry,
27 //     skip
28
29 //     for (auto& [v, w] : adj[u]) {
30 //         if (dist[u] + w < dist[v]) {
31 //             dist[v] = dist[u] + w;
32 //             pq.push({dist[v], v});
33 //         }
34 //     }
35
36 //     return dist; // dist[i] = shortest distance from
37 //     src to i (INF if unreachable)
38 // }

```

Build example:

```

1 int n = 5;
2 vector<vector<pair<int, long long>>> adj(n + 1); //
3 // 1-indexed, size n+1
4
5 auto addEdge = [&](int u, int v, long long w) {
6     adj[u].push_back({v, w});
7 };

```

```

6     adj[v].push_back({u, w}); // remove this line if
7 // the graph is directed
8 };
9
10 addEdge(1, 2, 4);
11 addEdge(1, 3, 1);
12 addEdge(3, 2, 2);
13
14 vector<long long> dist = dijkstra(n, adj, 1);
15 // dist[2] = 3 (via 1 -> 3 -> 2)

```

Bellman-Ford Algorithm (Handles Negative Weights + Cycle Detection)

```

1 #include <vector>
2 #include <climits>
3
4 using namespace std;
5
6 const long long INF = LLONG_MAX;
7
8 struct Edge {
9     int u, v;
10    long long w;
11 };
12
13 // Returns {dist, hasNegativeCycle}
14 // dist[i] = shortest distance from src to i (INF if
15 // unreachable)
16 pair<vector<long long>, bool> bellmanFord(int n,
17 // vector<Edge>& edges, int src) {
18 //     vector<long long> dist(n + 1, INF);
19 //     dist[src] = 0;
20
21 // Relax all edges V-1 times
22 for (int i = 0; i < n - 1; i++) {
23     for (auto& e : edges) {
24         if (dist[e.u] != INF && dist[e.u] + e.w
25             < dist[e.v]) {
26             dist[e.v] = dist[e.u] + e.w;
27         }
28     }
29 }
30
31 // Check for negative weight cycle: if any edge
32 // can still be relaxed
33 bool hasNegativeCycle = false;
34 for (auto& e : edges) {
35     if (dist[e.u] != INF && dist[e.u] + e.w <
36         dist[e.v]) {
37         hasNegativeCycle = true;
38         break;
39     }
40 }
41
42 return {dist, hasNegativeCycle};
43 }

```

Build example:

```

1 int n = 5;
2 vector<Edge> edges;
3
4 // Just push every edge into a flat vector, no
5 // grouping needed
6 edges.push_back({1, 2, 4});
7 edges.push_back({1, 3, 1});
8 edges.push_back({3, 2, 2});
9 edges.push_back({2, 4, -3}); // negative weight is
10 // fine

```

```

10 auto [dist, hasNegCycle] = bellmanFord(n, edges, 1);
11
12 if (hasNegCycle) {
13     // Negative cycle exists, shortest path is
14     //   ↪ undefined
15 } else {
16     // dist[4] = shortest distance
17 }

```

Floyd-Warshall Algorithm (All-Pairs Shortest Path)

```

1  #include <vector>
2  #include <climits>
3
4  using namespace std;
5
6  const long long INF = LLONG_MAX / 2; // Avoid
7  //   ↪ overflow when adding two INF values
8
9  // dist[i][j] initial state should be set by caller:
10 // dist[i][i] = 0, dist[i][j] = edge weight if edge
11 //   ↪ exists, else INF
12 // Returns true if graph has no negative cycle, false
13 //   ↪ otherwise
14 bool floydWarshall(int n, vector<vector<long long>>&
15 //   ↪ dist) {
16     for (int k = 1; k <= n; k++) {
17         for (int i = 1; i <= n; i++) {
18             for (int j = 1; j <= n; j++) {
19                 if (dist[i][k] + dist[k][j] <
20                     //   ↪ dist[i][j]) {
21                     dist[i][j] = dist[i][k] +
22                     //   ↪ dist[k][j];
23                 }
24             }
25         }
26     }
27
28     // Check for negative cycle: if any dist[i][i] <
29     //   ↪ 0
30     for (int i = 1; i <= n; i++) {
31         if (dist[i][i] < 0) return false; // Negative
32         //   ↪ cycle detected
33     }
34
35     return true; // No negative cycle, dist is valid
36 }

```

Build example:

```

1  int n = 4;
2  vector<vector<long long>> dist(n + 1, vector<long
3  //   ↪ long>(n + 1, INF));
4
5  // Initialize the diagonal
6  for (int i = 1; i <= n; i++) dist[i][i] = 0;
7
8  // Add edges (directed graph example)
9  dist[1][2] = 3;
10 dist[2][3] = 1;
11 dist[1][3] = 8; // if both a direct edge and a
12 //   ↪ shorter indirect path exist, the algorithm
13 //   ↪ resolves it automatically
14 dist[3][4] = 2;
15
16 bool ok = floydWarshall(n, dist);
17
18 if (!ok) {
19     // Negative cycle exists
20 } else {
21     // dist[1][4] = 6 (via 1 -> 2 -> 3 -> 4 = 3+1+2)
22 }

```

```

19 }

```

Minimum Spanning Tree (Kruskal's Algorithm)

Description: Finds the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph. It uses the greedy approach: sorts all edges in non-decreasing order of their weight, and adds edges one by one if they don't form a cycle. A Disjoint Set Union (DSU) structure is used to check and manage cycles efficiently. **Time Complexity:** $O(E \log E + E\alpha(V))$ where E is the number of edges, V is the number of vertices, and α is the Inverse Ackermann function. **Space Complexity:** $O(V)$ for the DSU parent and rank arrays. **Usage:** Suitable for sparse graphs. Supports 1-indexed vertices by default.

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Representation of an edge in the graph
7  struct Edge {
8      int u, v; // u, v: The two endpoint vertices
9      //   ↪ (nodes) connected by this edge
10     long long w; // w: The weight (cost/distance) of
11     //   ↪ this edge
12     // Overload the < operator to sort edges by
13     //   ↪ weight
14     bool operator<(const Edge& other) const {
15         return w < other.w;
16     }
17 };
18
19 // Disjoint Set Union (DSU) / Union-Find structure
20 struct DSU {
21     vector<int> parent; // parent[i] stores the
22     //   ↪ parent/leader of element i
23     vector<int> rank; // rank[i] stores the
24     //   ↪ approximate depth of the tree rooted at i (for
25     //   ↪ Union by Rank optimization)
26
27     DSU(int n) { // n: The number of
28     //   ↪ vertices/elements in the graph
29         parent.resize(n + 1);
30         rank.resize(n + 1, 0);
31         for (int i = 0; i <= n; i++) {
32             parent[i] = i; // Initially, every vertex
33             //   ↪ is its own parent (disjoint set)
34         }
35
36         // Finds the representative (root leader) of the
37         //   ↪ set containing element i
38         int find(int i) {
39             if (parent[i] == i)
40                 return i;
41             return parent[i] = find(parent[i]); // Path
42             //   ↪ compression optimization
43         }
44
45         // Merges the set containing element i with the
46         //   ↪ set containing element j.
47         // Returns true if they were in different sets
48         //   ↪ and successfully merged; false if they were
49         //   ↪ already in the same set.
50         bool unite(int i, int j) {
51             int root_i = find(i);
52             int root_j = find(j);
53
54             if (root_i == root_j) return false;
55
56             if (rank[root_i] < rank[root_j])
57                 parent[root_i] = root_j;
58             else if (rank[root_i] > rank[root_j])
59                 parent[root_j] = root_i;
60             else
61                 parent[root_j] = root_i;
62             rank[root_i]++;
63         }
64     };
65 }

```

```

41     if (root_i != root_j) {
42         // Union by rank optimization: attach the
43         ↪ smaller tree under the larger tree
44         if (rank[root_i] < rank[root_j]) {
45             parent[root_i] = root_j;
46         } else if (rank[root_i] > rank[root_j]) {
47             parent[root_j] = root_i;
48         } else {
49             parent[root_j] = root_i;
50             rank[root_i]++;
51         }
52         return true; // Successfully merged
53     }
54     return false; // Already in the same set
55     ↪ (connecting them would form a cycle)
56 }
57 // Computes the MST weight and optionally stores the
58 ↪ selected edges.
59 // n: Number of vertices (nodes) in the graph.
60 // edges: Input vector containing all edges in the
61 ↪ graph (will be sorted in-place).
62 // mstEdges: Output vector to store the selected
63 ↪ edges that form the MST.
64 // Returns the total MST weight, or -1 if the graph
65 ↪ is disconnected (no MST exists).
66 long long kruskal(int n, vector<Edge>& edges,
67 ↪ vector<Edge>& mstEdges) {
68     mstEdges.clear();
69     sort(edges.begin(), edges.end()); // Sort edges
70     ↪ in ascending order of weight (greedy approach)
71
72     DSU dsu(n); // Disjoint Set Union to detect
73     ↪ cycles
74     long long mst_weight = 0; // Accumulator for the
75     ↪ total weight of the MST
76     int edges_used = 0; // Counter for the number of
77     ↪ edges added to the MST
78
79     for (const auto& edge : edges) {
80         // Try to connect the two endpoints of the
81         ↪ current edge (edge.u and edge.v)
82         if (dsu.unite(edge.u, edge.v)) {
83             mst_weight += edge.w;
84             mstEdges.push_back(edge);
85             edges_used++;
86             // A tree with n vertices always has
87             ↪ exactly n - 1 edges
88             if (edges_used == n - 1) {
89                 break;
90             }
91         }
92     }
93
94     if (edges_used == n - 1) {
95         return mst_weight;
96     }
97     return -1; // Graph is not fully connected (could
98     ↪ not select n - 1 edges)
99 }

```

Cycle Detection

Description: General utility to check if a directed or undirected graph contains any cycles using DFS with a recursion stack (directed) or a visited-parent check (undirected). **Time Complexity:** $O(V + E)$ **Space Complexity:** $O(V)$

```

1  #include <vector>
2  using namespace std;
3
4  // Directed graph cycle detection via DFS
5  struct CycleDetector {
6      int n;
7      vector<vector<int>>> adj;
8      vector<int> color; // 0 = unvisited, 1 =
9      ↪ in-stack, 2 = done
10
11      CycleDetector(int n) : n(n), adj(n), color(n, 0)
12      ↪ {}
13
14      void addEdge(int u, int v) { adj[u].push_back(v);
15      ↪ }
16
17      bool dfs(int u) {
18          color[u] = 1;
19          for (int v : adj[u]) {
20              if (color[v] == 1) return true; // Back
21              ↪ edge -> cycle
22              if (color[v] == 0 && dfs(v)) return true;
23          }
24          color[u] = 2;
25          return false;
26      }
27
28      // Returns true if a cycle exists in the directed
29      ↪ graph
30      bool hasCycle() {
31          for (int i = 0; i < n; ++i) {
32              if (color[i] == 0 && dfs(i)) return true;
33          }
34          return false;
35      }
36 }
37
38 int main() {
39     CycleDetector graph(4);
40     // Add edges
41     graph.addEdge(0, 1);
42     graph.addEdge(1, 2);
43     graph.addEdge(2, 0); // Cycle 0 -> 1 -> 2 -> 0
44     graph.addEdge(2, 3);
45
46     if (graph.hasCycle()) {
47         cout << "Graph contains a cycle!" << endl;
48     } else {
49         cout << "Graph does not contain a cycle." <<
50         ↪ endl;
51     }
52
53     return 0;
54 }

```

Topological Sort (Kahn's Algorithm)

Description: Implements Topological Sort for a Directed Acyclic Graph (DAG) using Kahn's BFS-based algorithm (in-degree approach). **Time Complexity:** $O(V + E)$ **Space Complexity:** $O(V + E)$ **Usage:** Returns vertices in topological order. If the result size $< V$, a cycle exists.

```

1  #include <vector>
2  #include <queue>
3  using namespace std;
4
5  // Returns topological order of vertices 0..n-1, or
6  ↪ empty vector if cycle detected

```

```

6 vector<int> topoSort(int n, vector<vector<int>>& adj)
7 {
8     vector<int> indegree(n, 0);
9     for (int u = 0; u < n; ++u)
10         for (int v : adj[u]) indegree[v]++;
11
12     queue<int> q;
13     for (int i = 0; i < n; ++i)
14         if (indegree[i] == 0) q.push(i);
15
16     vector<int> order;
17     while (!q.empty()) {
18         int u = q.front(); q.pop();
19         order.push_back(u);
20         for (int v : adj[u]) {
21             if (--indegree[v] == 0) q.push(v);
22         }
23     }
24     return (int)order.size() == n ? order :
25         vector<int>{}; // empty if cycle
26 }
27
28 int main() {
29     int n = 4;
30     vector<vector<int>> adj(n);
31
32     // Add edges: 0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
33     adj[0].push_back(1);
34     adj[0].push_back(2);
35     adj[1].push_back(3);
36     adj[2].push_back(3);
37
38     vector<int> result = topoSort(n, adj);
39
40     if (result.empty()) {
41         cout << "Graph contains a cycle (No valid
42         topological order)." << endl;
43     } else {
44         cout << "Topological Sort Order: ";
45         for (int u : result) {
46             cout << u << " ";
47         }
48         cout << endl;
49     }
50
51     return 0;
52 }

```

Strongly Connected Components (Tarjan's Algorithm)

Description: Finds all Strongly Connected Components (SCCs) in a directed graph using Tarjan's single-pass DFS algorithm. **Time Complexity:** $O(V+E)$ **Space Complexity:** $O(V)$ **Usage:** Each SCC is a maximal set of vertices mutually reachable from each other.

```

1 #include <vector>
2 #include <stack>
3 using namespace std;
4
5 struct Tarjan {
6     int n, timer = 0;
7     vector<vector<int>> adj;
8     vector<int> disc, low, comp;
9     vector<bool> onStack;
10    stack<int> st;
11    int sccCount = 0;
12 }

```

```

13 Tarjan(int n) : n(n), adj(n), disc(n, -1),
14     low(n), comp(n, -1), onStack(n, false) {}
15
16 void addEdge(int u, int v) { adj[u].push_back(v); }
17
18 void dfs(int u) {
19     disc[u] = low[u] = timer++;
20     st.push(u);
21     onStack[u] = true;
22     for (int v : adj[u]) {
23         if (disc[v] == -1) {
24             dfs(v);
25             low[u] = min(low[u], low[v]);
26         } else if (onStack[v]) {
27             low[u] = min(low[u], disc[v]);
28         }
29     }
30     if (low[u] == disc[u]) {
31         while (true) {
32             int v = st.top(); st.pop();
33             onStack[v] = false;
34             comp[v] = sccCount;
35             if (v == u) break;
36         }
37         sccCount++;
38     }
39 }
40
41 // Returns comp[i] = SCC id of vertex i
42 // (0-indexed)
43 int solve() {
44     for (int i = 0; i < n; ++i)
45         if (disc[i] == -1) dfs(i);
46     return sccCount;
47 }
48
49 int main() {
50     // Graph: 0 -> 1 -> 2 -> 0 (one SCC), 2 -> 3 (3
51     // is its own SCC)
52     Tarjan tj(4);
53     tj.addEdge(0, 1);
54     tj.addEdge(1, 2);
55     tj.addEdge(2, 0); // Cycle: forms SCC {0, 1, 2}
56     tj.addEdge(2, 3); // 3 has no back edge, forms
57     // SCC {3}
58
59     int numSCC = tj.solve();
60     cout << "Number of SCCs: " << numSCC << endl; //
61     // 2
62
63     // Print SCC id of each vertex
64     for (int i = 0; i < 4; ++i) {
65         cout << "Node " << i << " -> SCC " <<
66         tj.comp[i] << endl;
67     }
68
69     // Node 0, 1, 2 share the same SCC id; Node 3 has
70     // a different id
71
72     return 0;
73 }

```

Disjoint Set Union (DSU / Union-Find)

Description: A standalone, highly reusable DSU class supporting path compression and union by rank/size. Efficiently manages disjoint sets for connectivity queries. **Time Complexity:** $O(\alpha(N))$ per operation (nearly $O(1)$ amortized), where α is the inverse Ackermann function. **Space Complexity:** $O(N)$ **Usage:** Cycle detection, Kruskal's MST,

connected components, etc.

```

1  #include <vector>
2  using namespace std;
3
4  struct DSU {
5      vector<int> parent, sz;
6
7      DSU(int n) : parent(n + 1), sz(n + 1, 1) {
8          for (int i = 0; i <= n; ++i) parent[i] = i;
9      }
10
11     int find(int x) {
12         if (parent[x] != x) parent[x] =
13             ↪ find(parent[x]); // Path compression
14         return parent[x];
15     }
16
17     // Returns true if x and y were in different sets
18     ↪ (successfully merged)
19     bool unite(int x, int y) {
20         x = find(x); y = find(y);
21         if (x == y) return false;
22         if (sz[x] < sz[y]) swap(x, y); // Union by
23         ↪ size
24         parent[y] = x;
25         sz[x] += sz[y];
26         return true;
27     }
28
29     bool connected(int x, int y) { return find(x) ==
30         ↪ find(y); }
31 };
32
33 int main() {
34     DSU dsu(5);
35     dsu.unite(0, 1);
36     dsu.unite(1, 2);
37     dsu.unite(3, 4);
38
39     cout << dsu.connected(0, 2) << endl; // 1 (true:
40     ↪ 0-1-2 are in the same set)
41     cout << dsu.connected(0, 3) << endl; // 0 (false:
42     ↪ {0,1,2} and {3,4} are separate)
43
44     dsu.unite(2, 3);
45     cout << dsu.connected(0, 4) << endl; // 1 (true:
46     ↪ all merged now)
47
48     return 0;
49 }

```

Lowest Common Ancestor (Binary Lifting)

Description: Preprocesses a rooted tree to answer LCA queries in $O(\log N)$ using binary lifting. Stores 2^k -th ancestors for each node. **Time Complexity:** $O(N \log N)$ preprocessing, $O(\log N)$ per query. **Space Complexity:** $O(N \log N)$ **Usage:** LCA of two nodes in a rooted tree; commonly used in path queries and tree DP.

```

1  #include <vector>
2  using namespace std;
3
4  const int MAXLOG = 18; // Supports up to  $2^{18} =$ 
5  ↪ 262144 nodes
6
7  struct LCA {
8      int n, LOG;
9      vector<int> depth;

```

```

9      vector<vector<int>> up; // up[v][k] =  $2^k$ -th
10     ↪ ancestor of v
11     vector<vector<int>> adj;
12
13     LCA(int n) : n(n), LOG(MAXLOG), depth(n + 1, 0),
14     ↪ up(n + 1, vector<int>(MAXLOG, 0)), adj(n + 1) {}
15
16     void addEdge(int u, int v) {
17         adj[u].push_back(v);
18         adj[v].push_back(u);
19     }
20
21     void dfs(int u, int p) {
22         up[u][0] = p;
23         for (int k = 1; k < LOG; ++k)
24             up[u][k] = up[up[u][k - 1]][k - 1];
25         for (int v : adj[u]) {
26             if (v != p) {
27                 depth[v] = depth[u] + 1;
28                 dfs(v, u);
29             }
30         }
31     }
32
33     void build(int root = 1) { dfs(root, root); }
34
35     int query(int u, int v) {
36         if (depth[u] < depth[v]) swap(u, v);
37         int diff = depth[u] - depth[v];
38         for (int k = 0; k < LOG; ++k)
39             if ((diff >> k) & 1) u = up[u][k];
40         if (u == v) return u;
41         for (int k = LOG - 1; k >= 0; --k)
42             if (up[u][k] != up[v][k]) { u =
43             ↪ up[u][k]; v = up[v][k]; }
44         return up[u][0];
45     }
46 };
47
48 int main() {
49     // Tree structure (1-indexed):
50     //      1
51     //     / \
52     //    2  3
53     //   / \
54     //  4  5
55     LCA lca(5);
56     lca.addEdge(1, 2);
57     lca.addEdge(1, 3);
58     lca.addEdge(2, 4);
59     lca.addEdge(2, 5);
60     lca.build(1); // Root at node 1
61
62     cout << lca.query(4, 5) << endl; // 2 (LCA of 4
63     ↪ and 5 is 2)
64     cout << lca.query(4, 3) << endl; // 1 (LCA of 4
65     ↪ and 3 is 1)
66     cout << lca.query(2, 4) << endl; // 2 (LCA of 2
67     ↪ and 4 is 2)
68
69     return 0;
70 }

```

Bipartite Matching (Hungarian Algorithm)

Finds the maximum matching in a bipartite graph. Nodes in set U are indexed 0 to $n - 1$. Nodes in set V are indexed 0 to $m - 1$.

```

1  #include <vector>
2

```

```

3 struct Hungarian {
4     int n, m;
5     std::vector<std::vector<int>> adj; // adj[u]
6     // contains neighbors in V
7     std::vector<int> match_v; // match_v[v]
8     // stores the matched u node
9     std::vector<bool> visited;
10
11     Hungarian(int n, int m) : n(n), m(m), adj(n),
12     // match_v(m, -1), visited(m, false) {}
13
14     void addEdge(int u, int v) {
15         adj[u].push_back(v);
16     }
17
18     // DFS to find an augmenting path
19     bool dfs(int u) {
20         for (int v : adj[u]) {
21             if (visited[v]) continue;
22             visited[v] = true;
23
24             // If v is not matched, or its match can
25             // find another choice
26             if (match_v[v] == -1 || dfs(match_v[v]))
27                 {
28                     match_v[v] = u;
29                     return true;
30                 }
31             return false;
32         }
33     }
34
35     // Returns the maximum matching size
36     int solve() {
37         int ans = 0;
38         std::fill(match_v.begin(), match_v.end(),
39         // -1);
40         for (int i = 0; i < n; ++i) {
41             std::fill(visited.begin(), visited.end(),
42             // false);
43             if (dfs(i)) ans++;
44         }
45         return ans;
46     }
47 };

```

9. Sequence Algorithms

Longest Increasing Subsequence (LIS)

Finds the length of the longest strictly increasing subsequence in $O(N \log N)$ time and $O(N)$ space.

```

1 #include <vector>
2 #include <algorithm>
3
4 // Returns the length of the LIS
5 int getLIS(const std::vector<int>& nums) {
6     if (nums.empty()) return 0;
7
8     std::vector<int> v;
9     for (int x : nums) {
10         auto it = std::lower_bound(v.begin(),
11         // v.end(), x);
12         if (it == v.end()) {
13             v.push_back(x); // x is larger than all
14             // elements, extend the sequence
15         } else {
16             *it = x; // Replace the first element >=
17             // x to maintain a smaller tail
18         }
19     }
20 }

```

```

16 }
17 return v.size();
18 }

```

10. Trie (Prefix Tree)

Description: A Trie (Prefix Tree) efficiently stores a collection of strings by sharing common prefixes. It supports insertion, exact string lookup, and prefix queries. **Time Complexity:** $O(L)$ for insert, search, and startsWith, L : The length of the string. **Space Complexity:** $O(\Sigma L)$, ΣL : The total number of inserted characters.

```

1 #include <vector>
2 #include <string>
3
4 using namespace std;
5
6 struct Trie {
7     // Each Trie node stores:
8     // nxt[i] : the index of the child corresponding
9     // to character ('a' + i)
10    // isEnd : true if a word ends at this node
11    struct Node {
12        int nxt[26];
13        bool isEnd;
14
15        Node() {
16            fill(nxt, nxt + 26, -1);
17            isEnd = false;
18        }
19    };
20
21    vector<Node> trie;
22
23    Trie() {
24        trie.emplace_back(); // Node 0 is the root
25    }
26
27    // Inserts a string into the Trie.
28    void insert(const string& word) {
29        int current = 0;
30
31        for (char ch : word) {
32            int index = ch - 'a';
33
34            // Create a new node if the path does not
35            // exist
36            if (trie[current].nxt[index] == -1) {
37                trie[current].nxt[index] =
38                // trie.size();
39                trie.emplace_back();
40            }
41
42            current = trie[current].nxt[index];
43        }
44
45        trie[current].isEnd = true;
46    }
47
48    // Returns true if the exact word exists in the
49    // Trie.
50    bool search(const string& word) {
51        int current = 0;
52
53        for (char ch : word) {
54            int index = ch - 'a';
55
56            if (trie[current].nxt[index] == -1)
57                return false;
58        }
59    }
60 }

```

```

54         current = trie[current].nxt[index];
55     }
56
57     return trie[current].isEnd;
58 }
59
60 // Returns true if there exists any word with the
61 // given prefix.
62 bool startsWith(const string& prefix) {
63     int current = 0;
64
65     for (char ch : prefix) {
66         int index = ch - 'a';
67
68         if (trie[current].nxt[index] == -1)
69             return false;
70
71         current = trie[current].nxt[index];
72     }
73
74     return true;
75 }
76 };

```

11. Aho-Corasick Automaton (AC Automaton)

Description: An extension of Trie that supports efficient **multiple pattern matching** by adding failure links (similar to KMP). It can find all occurrences of multiple patterns in a text in linear time.

Use AC Automaton when:

- Matching **multiple patterns** in a single text
- Sensitive word filtering
- Dictionary matching
- DNA sequence matching
- String DP (Automaton DP)

If there is only **one pattern**, KMP is usually sufficient.

Algorithm Steps

```

1 Patterns -> Build Trie -> Build Failure Links (BFS)
  ↳ -> Scan Text

```

Failure links allow the automaton to continue matching from the **longest valid suffix** instead of restarting from the root.

```

1 struct AhoCorasick {
2     static constexpr int SIGMA = 26;
3
4     struct Node {
5         int nxt[SIGMA];
6         int fail;
7         vector<int> out;
8
9         Node() {
10             memset(nxt, 0, sizeof(nxt));
11             fail = 0;
12         }
13 };
14
15 vector<Node> trie;
16
17 AhoCorasick() {
18     trie.emplace_back(); // root
19 }

```

```

20
21 void insert(const string &s, int id) {
22     int cur = 0;
23     for (char c : s) {
24         int x = c - 'a';
25         if (!trie[cur].nxt[x]) {
26             trie[cur].nxt[x] = trie.size();
27             trie.emplace_back();
28         }
29         cur = trie[cur].nxt[x];
30     }
31     trie[cur].out.push_back(id);
32 }
33
34 void build() {
35     queue<int> q;
36
37     for (int c = 0; c < SIGMA; c++) {
38         int v = trie[0].nxt[c];
39         if (v) q.push(v);
40     }
41
42     while (!q.empty()) {
43         int u = q.front();
44         q.pop();
45
46         for (int c = 0; c < SIGMA; c++) {
47             int &v = trie[u].nxt[c];
48
49             if (v) {
50                 trie[v].fail =
51                 ↳ trie[trie[u].fail].nxt[c];
52                 trie[v].out.insert(
53                     trie[v].out.end(),
54                     ↳ trie[trie[v].fail].out.begin(),
55                     trie[trie[v].fail].out.end()
56                 );
57                 q.push(v);
58             } else {
59                 v = trie[trie[u].fail].nxt[c];
60             }
61         }
62     }
63
64     vector<int> query(const string &text) {
65         vector<int> res;
66         int cur = 0;
67
68         for (char c : text) {
69             cur = trie[cur].nxt[c - 'a'];
70             for (int id : trie[cur].out)
71                 res.push_back(id);
72         }
73
74         return res;
75     }
76 };

```

Procedure

1. Insert all patterns into the Trie.
2. Build failure links using BFS.
3. Scan the text once.
4. Report every matched pattern.

Notes

- Root index is usually 0.
- Failure links are built using **BFS**.

- Missing transitions are completed during `build()`, so matching is $O(|\text{Text}|)$.
- `out` stores all pattern IDs ending at the current node.
- Frequently combined with **DP** to solve forbidden-string and string-counting problems.

12. Expected Value DP

Idea: Define the expectation of each state and write the expectation equation directly.

General Formula

$$E(s) = \sum P_i (C_i + E(\text{next}_i))$$

where

- P_i : probability of transition
- C_i : immediate cost
- $E(\text{next}_i)$: future expectation

If Self-loop Exists

Suppose

$$E(s) = A + pE(s)$$

Move the self-loop term to the left:

$$E(s) = \frac{A}{1-p}$$

This is the most common transformation in Expected Value DP.

Steps

1. Define the DP state.
2. Enumerate all possible transitions.
3. Write the expectation equation directly.
4. Move self-loop terms to the left if necessary.
5. Solve the equation.
6. Use memoization or DP.

Common State Types

- `dp[i]`
- `dp[mask]`
- `dp[x][y]`
- `dp[a][b][c]`

13. Advanced Data Structures

Segment Tree with Lazy Propagation

Description: A robust Segment Tree supporting both **range updates** and **range queries** (e.g., Range Sum or Range Max) using lazy propagation to defer updates efficiently. **Time Complexity:** $O(\log N)$ per update and query. **Space Complexity:** $O(N)$ **Usage:** Ideal when both range updates and range queries are required simultaneously.

```
1 #include <vector>
2 using namespace std;
3
4 // Segment Tree with Lazy Propagation (Range Add,
5   Range Sum)
```

```
6 struct SegTree {
7     int n;
8     vector<long long> tree, lazy;
9
10     SegTree(int n) : n(n), tree(4 * n, 0), lazy(4 *
11         n, 0) {}
12
13     void build(vector<int>& a, int node, int start,
14         int end) {
15         if (start == end) {
16             tree[node] = a[start];
17         } else {
18             int mid = (start + end) / 2;
19             build(a, 2 * node, start, mid);
20             build(a, 2 * node + 1, mid + 1, end);
21             tree[node] = tree[2 * node] + tree[2 *
22                 node + 1];
23         }
24     }
25
26     void pushDown(int node, int start, int end) {
27         if (lazy[node] != 0) {
28             int mid = (start + end) / 2;
29             tree[2 * node] += lazy[node] * (mid
30                 - start + 1);
31             tree[2 * node + 1] += lazy[node] * (end
32                 - mid);
33             lazy[2 * node] += lazy[node];
34             lazy[2 * node + 1] += lazy[node];
35             lazy[node] = 0;
36         }
37     }
38
39     // Range update: add val to all elements in [l,
40         r]
41     void update(int node, int start, int end, int l,
42         int r, long long val) {
43         if (r < start || end < l) return;
44         if (l <= start && end <= r) {
45             tree[node] += val * (end - start + 1);
46             lazy[node] += val;
47             return;
48         }
49         pushDown(node, start, end);
50         int mid = (start + end) / 2;
51         update(2 * node, start, mid, l, r, val);
52         update(2 * node + 1, mid + 1, end, l, r,
53             val);
54         tree[node] = tree[2 * node] + tree[2 * node +
55             1];
56     }
57
58     // Range query: sum of elements in [l, r]
59     long long query(int node, int start, int end,
60         int l, int r) {
61         if (r < start || end < l) return 0;
62         if (l <= start && end <= r) return
63             tree[node];
64         pushDown(node, start, end);
65         int mid = (start + end) / 2;
66         return query(2 * node, start, mid, l, r) +
67             query(2 * node + 1, mid + 1, end, l,
68                 r);
69     }
70
71     // Convenience wrappers (1-indexed)
72     void update(int l, int r, long long val) {
73         update(1, 1, n, l, r, val);
74     }
75     long long query(int l, int r) { return query(1,
76         1, n, l, r); }
77 };
```

```

63 int main() {
64     // Array: [1, 2, 3, 4, 5] (1-indexed)
65     vector<int> a = {0, 1, 2, 3, 4, 5}; // a[0]
66     ↪ unused
67     SegTree st(5);
68     st.build(a, 1, 1, 5);
69
70     cout << st.query(1, 5) << endl; // 15 (sum of
71     ↪ all)
72     cout << st.query(2, 4) << endl; // 9 (sum of
73     ↪ a[2..4] = 2+3+4)
74
75     st.update(2, 4, 10); // Add 10 to each element in
76     ↪ [2, 4]
77     // Array is now: [1, 12, 13, 14, 5]
78
79     cout << st.query(1, 5) << endl; // 45 (15 + 10*3)
80     cout << st.query(2, 4) << endl; // 39 (12+13+14)
81
82     return 0;
83 }

```